Cairo University
Faculty of Engineering

Department of Computer
Engineering

ELC 325B – Spring 2025

Digital Communications

Assignment #2

Matched Filters

Submitted to

Dr. Mai Badaway

Dr. Hala Abdelkader

Eng. Mohammed Khalid

Submitted by

Name	Sec	BN
Ahmed Hamdy Mohammed	1	4
Abdelrahman Mostafa Gomaa	1	31

Contents

Part 1: Handwritten Analysis **3**

Part II: Simulation **5**

Hand Analysis 5

Input function plot 7

BER Theoretical and Simulation For The Three Filters 10

Comment 10

Is the BER an increasing or a decreasing function of E/No? Why? 10

Which case has the lowest BER? Why? 10

Code **11**

Part 1: Handwritten Analysis

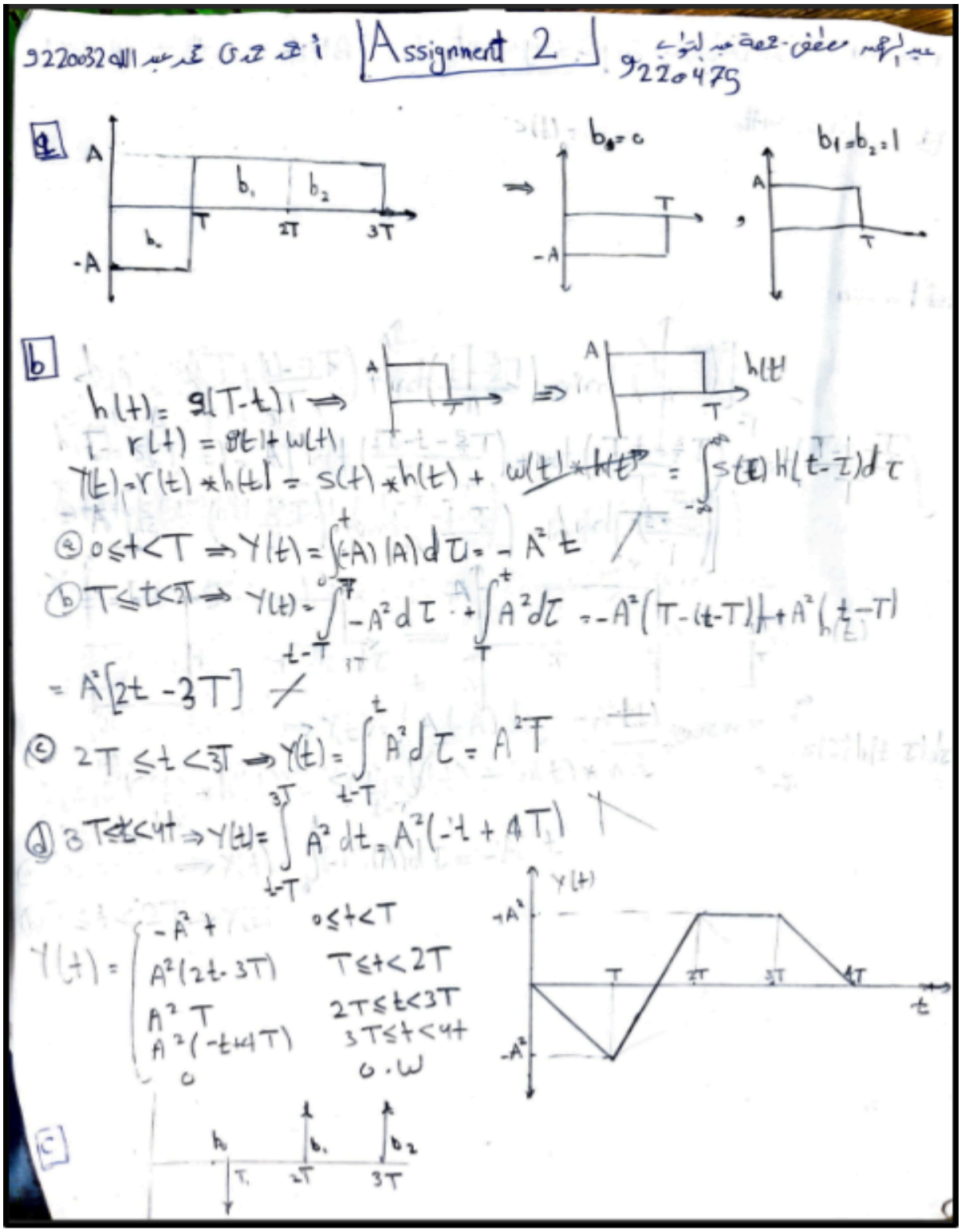


Figure 1 Handwritten Analysis Part I (a,b,c)

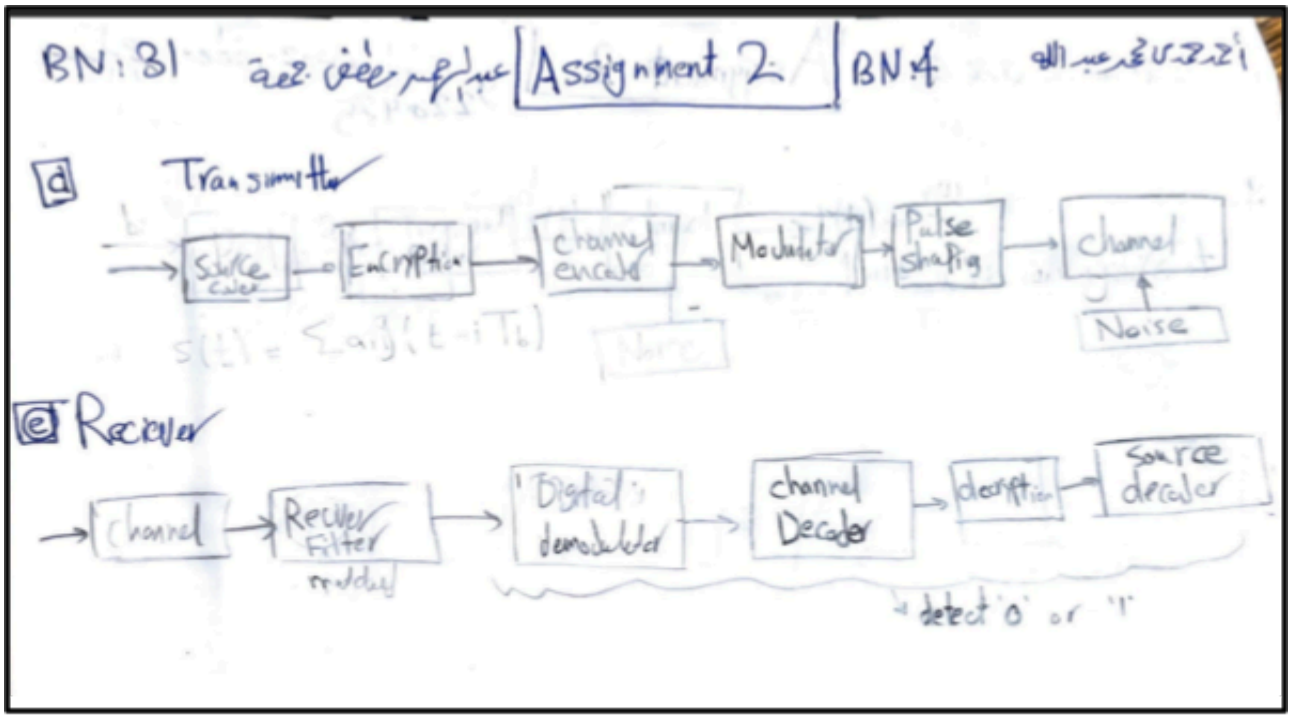


Figure 2 Handwritten Analysis Part I (d,e)

Part II: Simulation

Hand Analysis

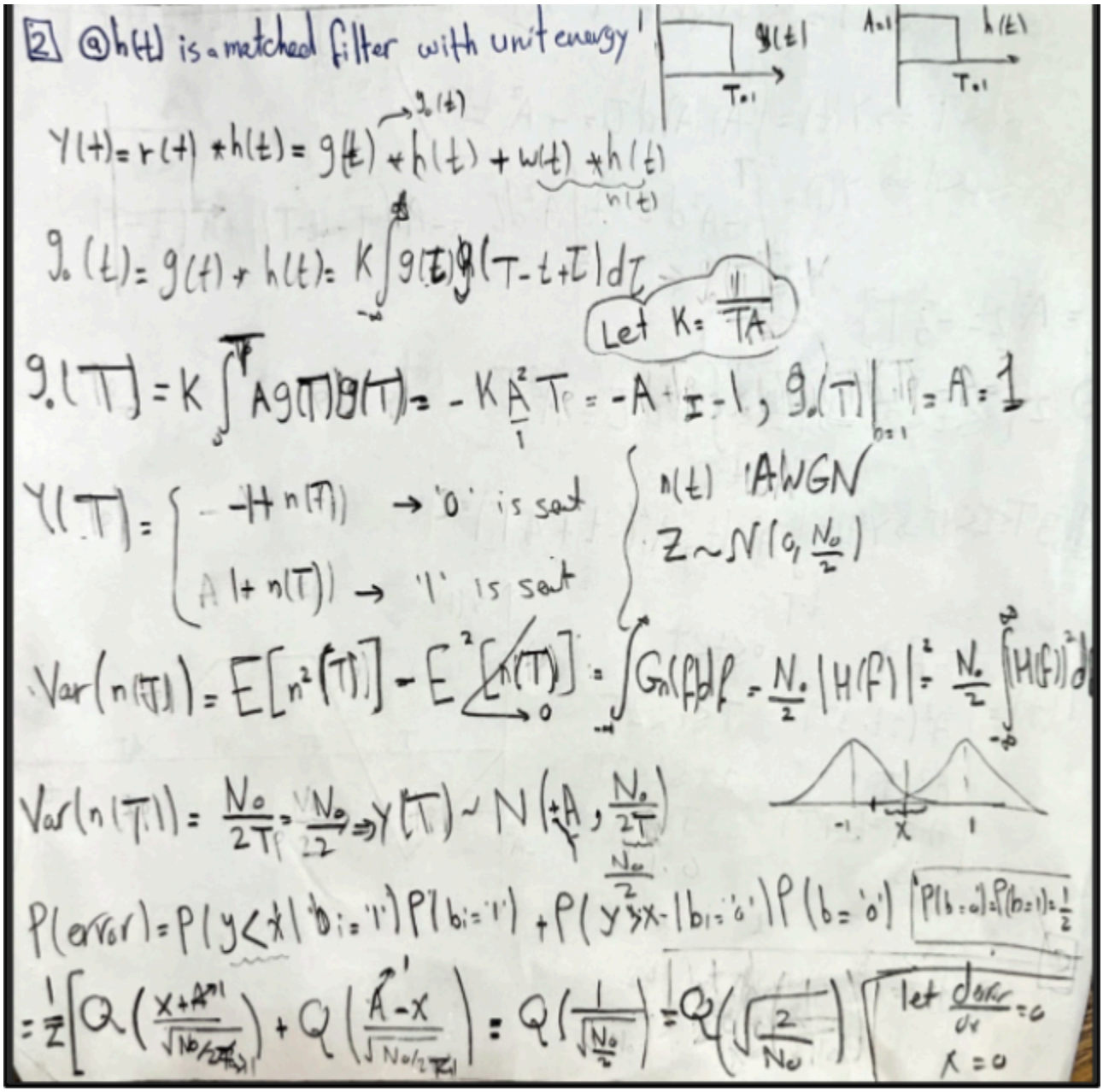


Figure 3 Handwritten Analysis Part II (case a)

BN:31 | Assignment 2 | BN:4

(b) $h(t)$ is not existent (i.e $h(t)=\delta(t)$)

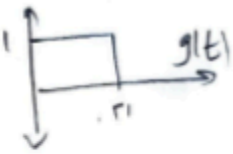
$$Y(t)=h(t)+r(t)=h(t)*g(t)+h(t)*w(t)=g(t)+w(t)$$

$$Y(T)=\begin{cases}-A+w(t) & b='0' \\ A+w(t) & b='1'\end{cases} \Rightarrow Y(t)\sim N(\pm A,\frac{N_0}{2})$$

$$P(error)=P(Y<x\mid b='1')P(b='1')+P(Y>x\mid b='0')P(b='0')=Q\left(\frac{1}{\sqrt{N_0/2}}\right)$$ Same as (a)

(c) $h(t)$ has the following impulse response

$$y(t)=\underbrace{g(t)*h(t)}_{g_o(t)}+\underbrace{w(t)*h(t)}_{n(t)}$$



$$g_o(t)\Big|_{t=T}=\int_0^T h(t-\tau)g(\tau)d\tau=\int_0^1(\sqrt{3}-\sqrt{3}\tau)d\tau=\frac{\sqrt{3}}{2}A=\frac{\sqrt{3}}{2}$$

$$g(T)\sim\begin{cases}-\frac{\sqrt{3}}{2}+n(T) & b='0' \\ \frac{\sqrt{3}}{2}+n(T) & b='1'\end{cases} \Rightarrow Y(T)\sim N\left(\pm\frac{\sqrt{3}}{2},\frac{N_0}{2}\right)$$

$$E[n^2(t)]=\int_{-\infty}^{\infty}G_f(f)df=\frac{N_0}{2}\int_{-\infty}^{\infty}|H(f)|^2df=\frac{N_0}{2}$$

$$P(error)=P(Y<x\mid b='1')P(b='1')+P(Y>x\mid b='0')P(b='0')=Q\left(\frac{\frac{\sqrt{3}}{2}}{\sqrt{N_0/2}}\right)$$
$$=Q\left(\sqrt{\frac{3}{N_0}}\right)$$

For all cases we assumed that $P(b='1')=P(b='0')=\frac{1}{2}$

$$\frac{\partial P_{error}}{\partial x}=0 \Rightarrow x_{opt}=0$$

the optimal x

Figure 4 Handwritten Analysis Part II (case b,c)

Input function plot

We use the following parameters:

- Input signal pulse shape: [0,1,1,0,1,1,0,1]
- samples = 50

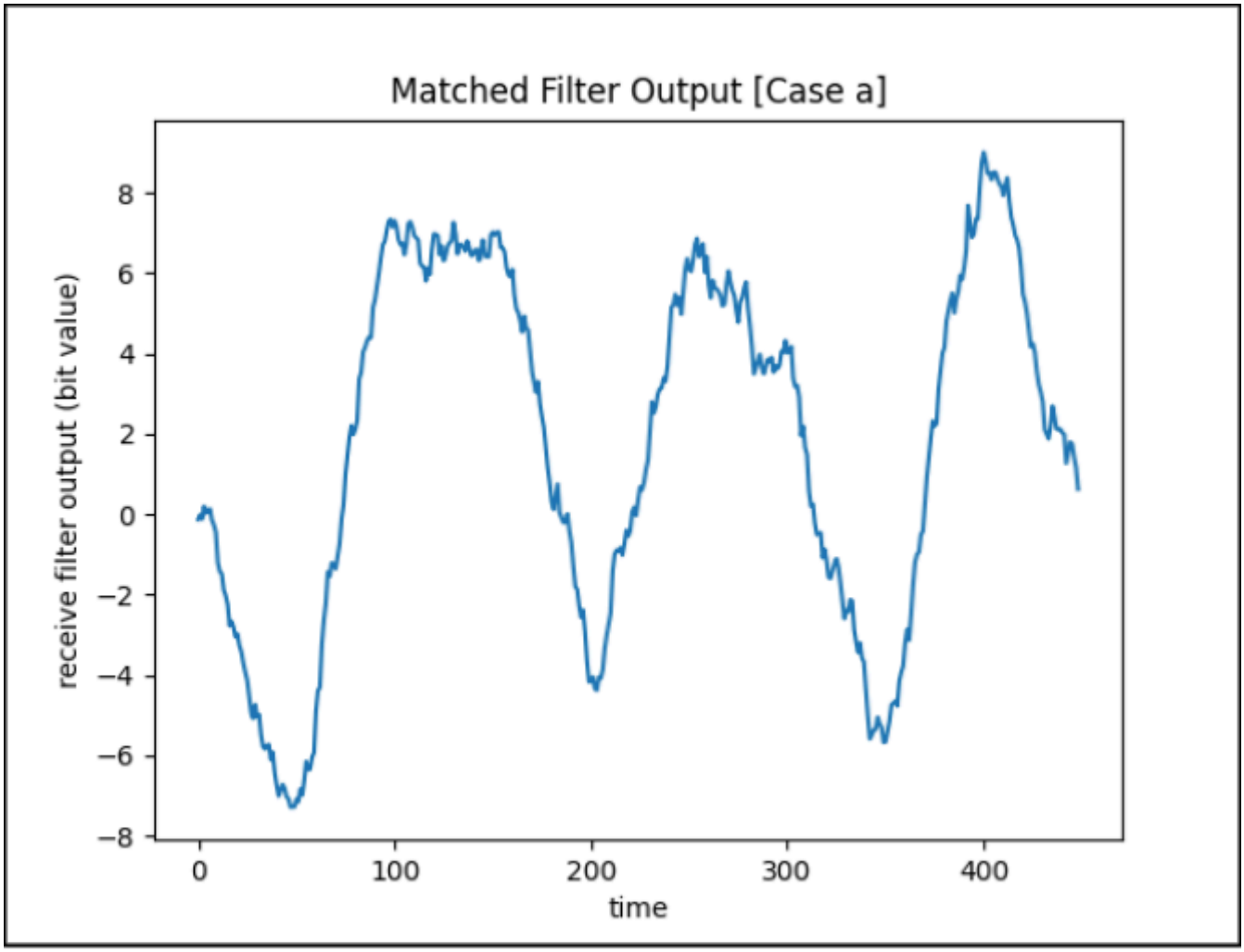


Figure 5 Matched Filter Output

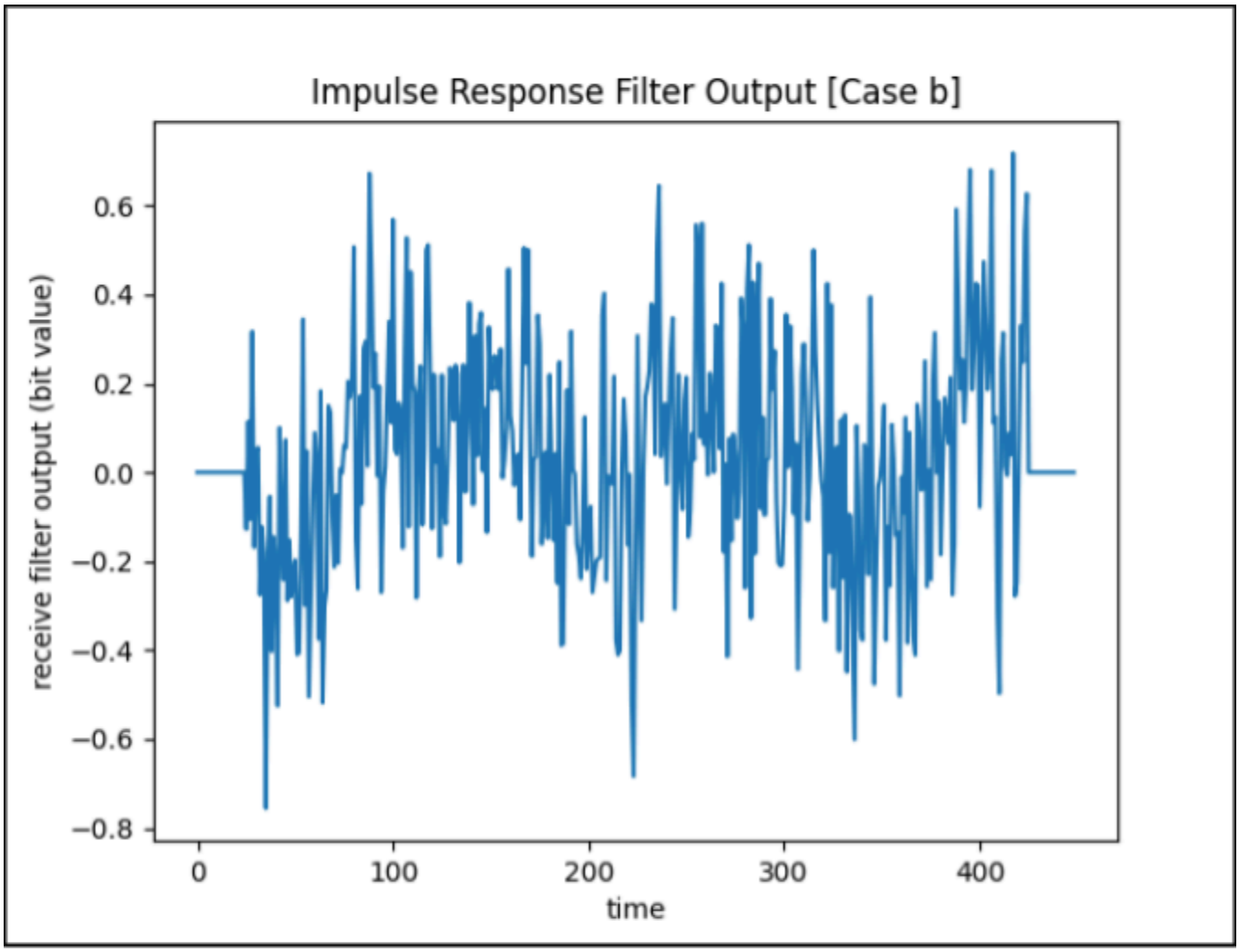


Figure 6 Impulse Filter Output

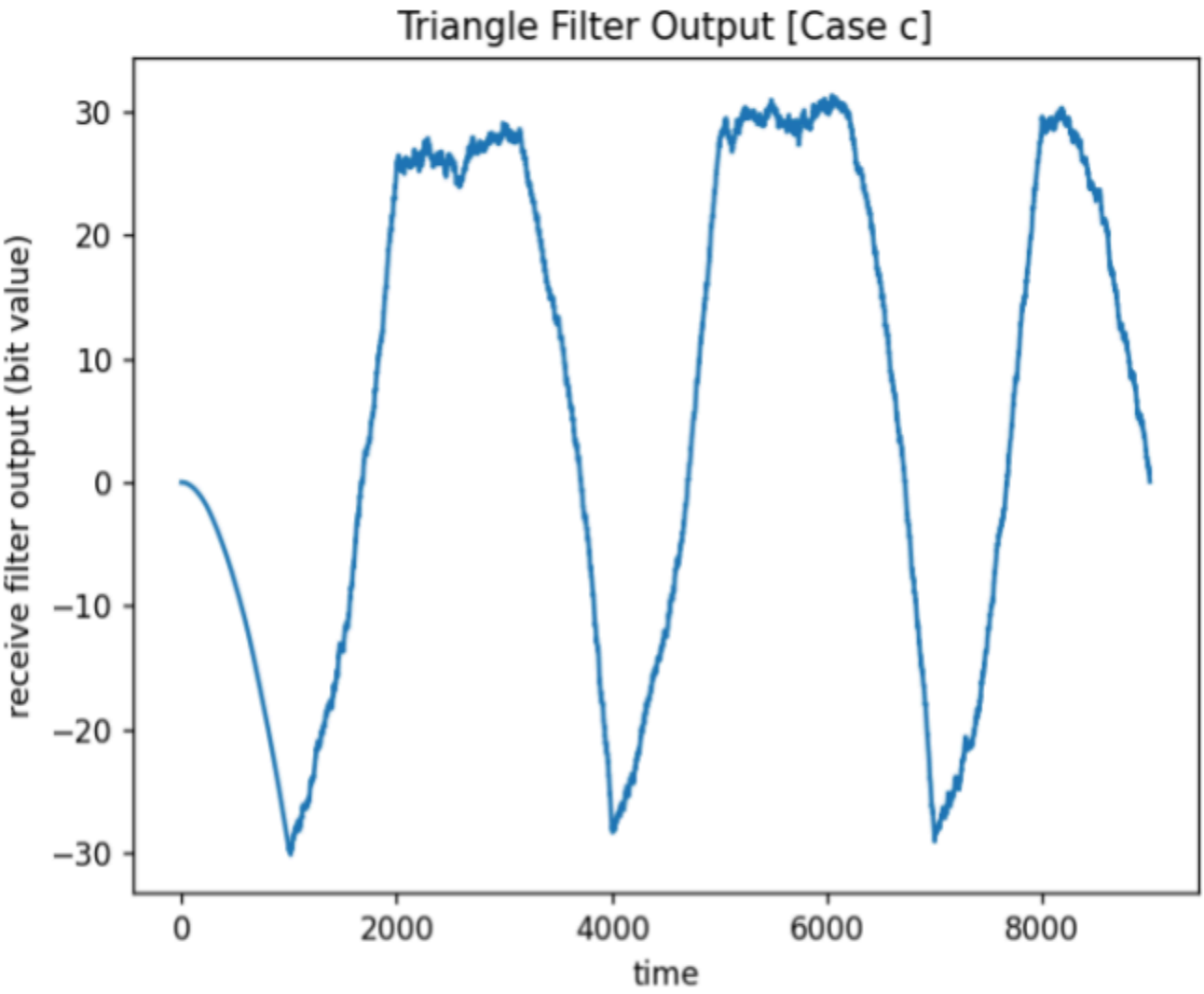


Figure 7 Triangle Filter Output

BER Theoretical and Simulation For The Three Filters

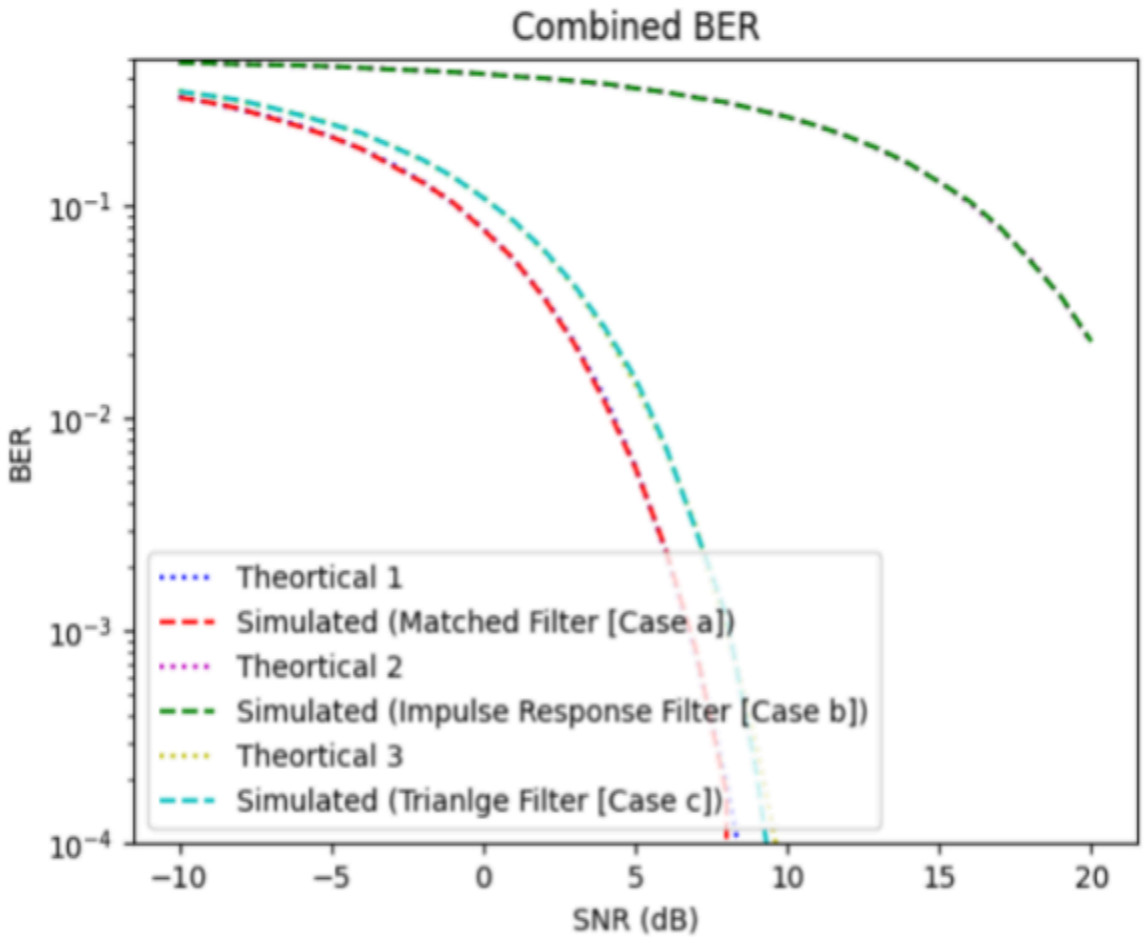


Figure 8 BER Theoretical vs Simulated

Comment

Plotting theoretical and simulated BER shows that the simulation of each filter almost follows the theoretical curve of its filter.

Is the BER an increasing or a decreasing function of E/No? Why?

BER is a decreasing function of E/No.
If the ratio between signal energy and noise energy is high, the noise will not significantly affect the signal. Hence, BER will decrease.

Which case has the lowest BER? Why?

The matched filter is the one with the lowest BER. The matched filter is specifically designed to maximize the peak SNR of the signal. Hence, a low BER.

Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import convolve
from scipy.special import erfc

# Helper functions
def pulse_shape(bits_number, samples_number, bits):
    input_signal = np.ones((bits_number, samples_number))
    for i in range(bits_number):
        if bits[i] == 0:
            input_signal[i,:] = -input_signal[i,:]
    return input_signal

def add_noise(bits_number, samples_number, input_signal, E, snr):
    sigma = np.sqrt(E/(2.0*snr))
    noise = np.random.normal(0, sigma, (1, bits_number*samples_number))
    input_with_noise = input_signal/np.sqrt(samples_number)

    for i in range(bits_number):
        input_with_noise[i,:] += noise[0,
samples_number*i:samples_number*(i+1)]

    return input_with_noise.reshape(-1, 1)
```

```
def sample(output, bits_number, samples_number):
    samples = np.ones(bits_number)
    for i in range(bits_number):
        samples[i] = output[(samples_number - 1) + samples_number * i] > 0
    return samples


# Parameters
samples_number = 50
bits = np.array([0,1,1,0,1,1,0,1])
bits_number = len(bits)
E = 1
snr_range = np.arange(-10, 21, 1)
snr = 10**(10/10)
input_signal = pulse_shape(bits_number, samples_number, bits)
s = input_signal.reshape(-1, 1)
recieved_signal = add_noise(bits_number,samples_number,input_signal,E,snr)


# Define filters
matched_filter = np.ones(samples_number)
impulse_response_filter = np.zeros(samples_number)
impulse_response_filter[samples_number//2] = 1
tri_filter = (np.sqrt(3)/samples_number)*np.arange(samples_number)
filters = [matched_filter, impulse_response_filter, tri_filter]
outputs = []
```

```
# Apply filters
for filter in filters:
    outputs.append(convolve(recieved_signal.flatten(), filter))

# Plot filter outputs
titles = ['Matched Filter Output [Case a]', 'Impulse Response Filter
Output [Case b]', 'Triangle Filter Output [Case c]']
for output, title in zip(outputs, titles):
    plt.figure()
    plt.plot(output)
    plt.title(title)
    plt.xlabel('time')
    plt.ylabel('receive filter output (bit value)')
    plt.savefig(f'{title}.png')
    plt.show()

# BER vs SNR simulation
bits_number = 100000
bits = np.ones(bits_number)
indices = np.random.permutation(bits_number)[:1]
bits[indices] = 0
s = pulse_shape(bits_number, samples_number, bits)
BER_sim = np.zeros((3, len(snr_range)))
BER_theo = np.zeros((3, len(snr_range)))
```

```
for i, snr_db in enumerate(snr_range):
   snr = 10**(snr_db/10)
   recieved_signal = add_noise(bits_number, samples_number, s, E, snr)
   for k, f in enumerate(filters):
       output = convolve(recieved_signal.flatten(), f)
       output_samples = sample(output, bits_number, samples_number)
       BER_sim[k,i] = np.sum(output_samples != bits) / bits_number
   BER_theo[0,i] = 0.5*erfc(np.sqrt(snr))
   BER_theo[1,i] = 0.5*erfc(np.sqrt(snr/samples_number))
   BER_theo[2,i] = 0.5*erfc((np.sqrt(3)/2)*np.sqrt(snr))
# Plot combined BER
plt.figure()
plt.semilogy(snr_range, BER_theo[0], 'b:', label='Theortical 1')
plt.semilogy(snr_range, BER_sim[0], 'r--', label='Sim (Matched Filter)')
plt.semilogy(snr_range, BER_theo[1], 'm:', label='Theortical 2')
plt.semilogy(snr_range, BER_sim[1], 'g--', label='Sim (Impulse Filter)')
plt.semilogy(snr_range, BER_theo[2], 'y:', label='Theortical 3')
plt.semilogy(snr_range, BER_sim[2], 'c--', label='Sim (Triangle Filter)')
plt.ylim(1e-4, 0.5)
plt.xlabel('SNR (dB)')
plt.ylabel('BER')
plt.legend()
plt.title('Combined BER')
plt.savefig('combined_ber.png')
plt.show()
```